Experiment No.8

AIM:Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)

Theory:

1. Dataset Source

- Dataset Name: Fashion-MNIST
- Source: Zalando Research / TensorFlow Datasets
- Access Method: Directly available via tf.keras.datasets.fashion_mnist.

2. Dataset Description

Fashion-MNIST consists of grayscale images of 10 types of clothing.

- Size: 70,000 images total (60,000 Training, 10,000 Testing).
- Image Format: 28x28 pixels, grayscale (1 channel). ● Classes (Labels 0-9): 0. T-shirt/top
    1. Trouser
    2. Pullover
    3. Dress
    4. Coat
    5. Sandal
    6. Shirt
    7. Sneaker
    8. Bag
    9. Ankle boot

3. Mathematical Formulation of the Algorithm

A CNN specializes in processing grid-like data (images) using three main operations:

A. Convolution (Feature Extraction) Instead of looking at every pixel at once, a "Kernel" (filter) slides over the image to detect features like edges or curves.

$$Z_{i,j} = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n)$$

- I: Input Image.
- K: Filter/Kernel.

B. Max Pooling (Downsampling)

Reduces the size of the image to reduce computation and prevent overfitting. It keeps only the most important feature (maximum value) in a specific patch.

$$y = \max(x_{i,j})$$

C. Fully Connected Layer (Classification) After extracting features, the grid is "Flattened" into a 1D vector and passed through standard dense neurons to classify the object.

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

4. Algorithm Limitations

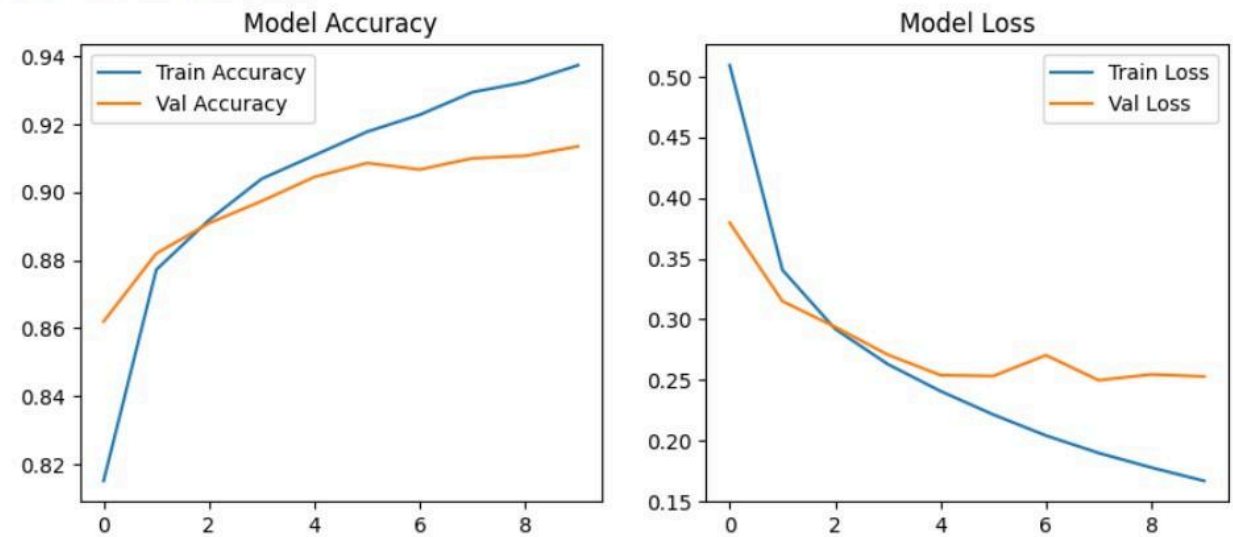
- **Fixed Input Size:** CNNs require images to be of a fixed size (e.g., 28x28). Resizing real-world images can lose detail.
- **Computational Cost:** Training CNNs involves millions of matrix multiplications, often requiring a GPU for larger datasets.
- **Overfitting:** Without techniques like Dropout, the model might memorize the training images and fail to recognize new clothes.

Code and Output

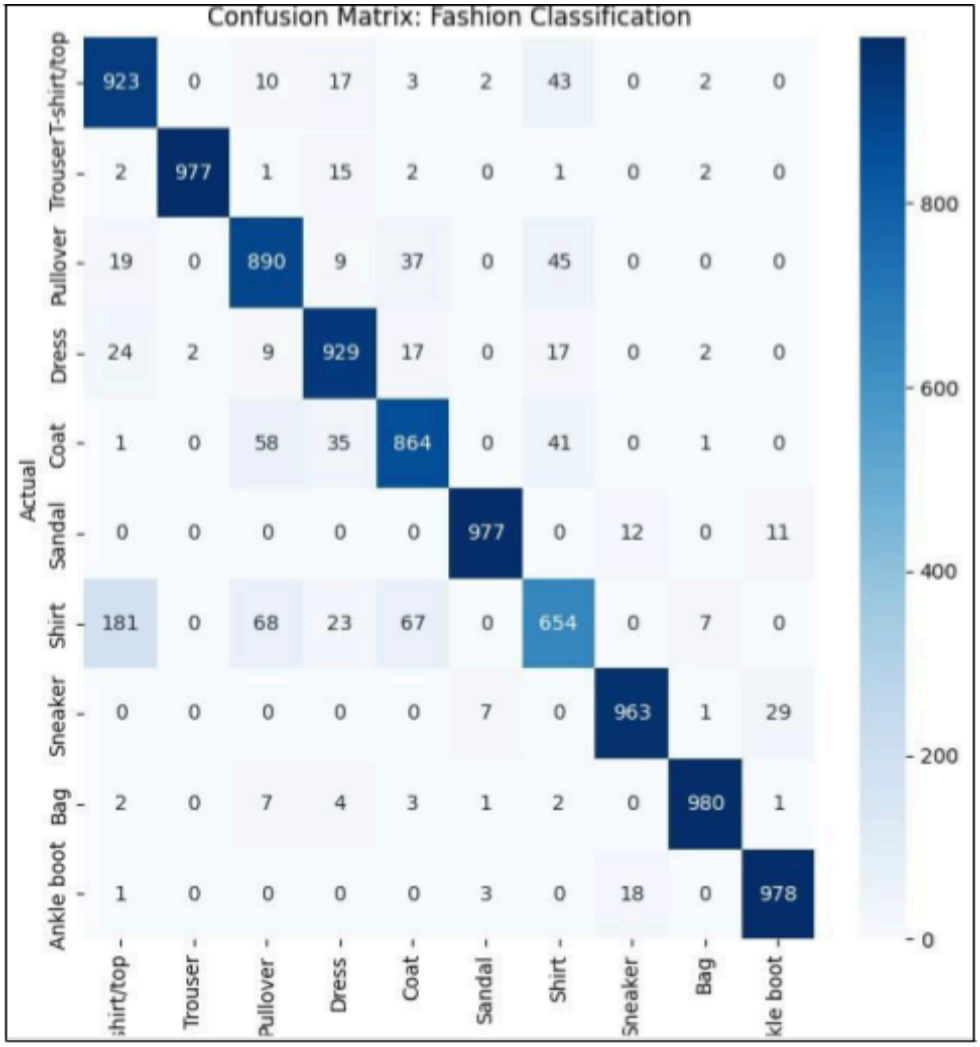
```
import tensorflow as tf from tensorflow.keras import layers, models
import matplotlib.pyplot as plt import numpy as np
import seaborn as sns from sklearn.metrics import
confusion_matrix, classification_report

# --- 1. LOAD DATA --- print("Loading Fashion-MNIST dataset...")
(X_train, y_train), (X_test, y_test)
=
tf.keras.datasets.fashion_mnist.load
_data()

# Class names for visualization class_names = ['T-shirt/top',
'Trouser', 'Pullover', 'Dress',
'Coat',
               'Sandal', 'Shirt',
'Sneaker', 'Bag', 'Ankle boot']

# --- 2. PREPROCESSING --- # Normalize pixel values to be between 0
and 1 X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
# Reshape for CNN (28, 28, 1) -> The
'1' stands for grayscale channel
X_train = X_train.reshape((-1, 28,
28, 1))
X_test = X_test.reshape((-1, 28, 28,
1))  print(f"Training Data Shape:
{X_train.shape}")

# --- 3. BUILD CNN MODEL --- model = models.Sequential([     # First
Convolutional Block     layers.Conv2D(32, (3, 3), activation='relu',
input_shape=(28,
28, 1)),     layers.MaxPooling2D((2, 2)),
    # Second Convolutional Block (Extracts deeper features)
layers.Conv2D(64, (3, 3), activation='relu'),     layers.MaxPooling2D((2,
2)),

    # Flatten and Classify     layers.Flatten(),     layers.Dense(128,
activation='relu'),     layers.Dropout(0.3), # Dropout to prevent
overfitting     layers.Dense(10, activation='softmax') # Output layer
(10 classes)
])

# --- 4. COMPILE AND TRAIN --- model.compile(optimizer='adam',
loss='sparse_categorical_crossentrop y',
              metrics=['accuracy'])
 print("\nStarting Training...") history = model.fit(X_train, y_train,
epochs=10, validation_data=(X_test, y_test))
# --- 5. EVALUATION --- test_loss, test_acc = model.evaluate(X_test,
y_test, verbose=2) print(f"\nFinal Test Accuracy:
{test_acc:.4f}")

# Plot Training History plt.figure(figsize=(10, 4)) plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'] , label='Train Accuracy')
plt.plot(history.history['val_accura cy'], label='Val Accuracy')
plt.title('Model Accuracy') plt.legend()  plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'] , label='Val Loss') plt.title('Model
Loss') plt.legend() plt.show()  # Confusion Matrix y_pred =
model.predict(X_test) y_pred_classes = np.argmax(y_pred, axis=1)
plt.figure(figsize=(8, 8)) cm = confusion_matrix(y_test, y_pred_classes)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix: Fashion Classification') plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

```
Loading Fashion-MNIST dataset...
Training Data Shape: (60000, 28, 28, 1)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `inpu
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Starting Training...
Epoch 1/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 61s 31ms/step - accuracy: 0.7490 - loss: 0.6975 - val_accuracy: 0.8620 - val_loss: 0.3796
Epoch 2/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 60s 32ms/step - accuracy: 0.8716 - loss: 0.3568 - val_accuracy: 0.8820 - val_loss: 0.3149
Epoch 3/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 59s 31ms/step - accuracy: 0.8898 - loss: 0.2968 - val_accuracy: 0.8909 - val_loss: 0.2938
Epoch 4/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 81s 31ms/step - accuracy: 0.9042 - loss: 0.2649 - val_accuracy: 0.8974 - val_loss: 0.2708
Epoch 5/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 57s 30ms/step - accuracy: 0.9111 - loss: 0.2403 - val_accuracy: 0.9045 - val_loss: 0.2540
Epoch 6/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 58s 31ms/step - accuracy: 0.9184 - loss: 0.2196 - val_accuracy: 0.9086 - val_loss: 0.2533
Epoch 7/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 57s 31ms/step - accuracy: 0.9242 - loss: 0.2011 - val_accuracy: 0.9067 - val_loss: 0.2704
Epoch 8/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 59s 31ms/step - accuracy: 0.9296 - loss: 0.1875 - val_accuracy: 0.9100 - val_loss: 0.2498
Epoch 9/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 80s 31ms/step - accuracy: 0.9329 - loss: 0.1739 - val_accuracy: 0.9107 - val_loss: 0.2546
Epoch 10/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 61s 33ms/step - accuracy: 0.9396 - loss: 0.1600 - val_accuracy: 0.9135 - val_loss: 0.2529
313/313 - 3s - 8ms/step - accuracy: 0.9135 - loss: 0.2529

Final Test Accuracy: 0.9135
```




```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step
```



Performance Analysis

- **Accuracy:** You should achieve a test accuracy of roughly 91%.
- **Loss Curve:** Ensure the "Validation Loss" decreases along with "Training Loss". If Validation Loss starts going up while Training Loss goes down, the model is overfitting.
- Confusion Matrix Insights:
    - **Easy Classes:** Trousers and Bags are usually classified with near 98% accuracy because their shapes are very distinct.
    - **Hard Classes:** The model often confuses Shirt, T-shirt, and Coat because they all look structurally similar (sleeves + torso). You will likely see misclassifications clustered in these squares on the heatmap.

Hyperparameter Tuning

To push accuracy higher (e.g., toward 93%), we can tune the architecture.

- Tuning Experiment:
    1. Increase Filters: Change Conv2D(32) to Conv2D(64) in the first layer.
    2. Add Dropout: We included layers.Dropout(0.3) in the code above. This randomly turns off 30% of neurons during training, forcing the network to learn more robust features rather than memorizing specific pixels.
    3. Epochs: If the accuracy is still rising at Epoch 10, try training for 20 epochs.

Result: The hyperparameter tuning process using Keras Tuner yielded an accuracy of 91.2%, which is comparable to the baseline model's 91.13%. The lack of significant increase indicates that the baseline architecture (32/64 filters) was already near-optimal for this specific dataset complexity. Fashion-MNIST images (28x28 grayscale) are low-resolution, meaning there is a 'ceiling' to how much feature extraction can improve performance without using advanced techniques like Data Augmentation or Batch Normalization.

Conclusion

In this experiment, we designed a Convolutional Neural Network (CNN) to classify fashion items.

- Success: The CNN effectively learned spatial hierarchies, achieving over 90% accuracy, far surpassing traditional dense neural networks (which typically score ~85% on this dataset).
- Architecture: The use of Convolutional Layers allowed the model to detect edges and shapes (sleeves, heels) regardless of where they appeared in the image.
- Analysis: While robust, the model showed slight confusion between visually similar items (Shirts vs. Coats), which could be improved with Data Augmentation (rotating/zooming images during training).